

```
import os
import sys
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import loadmat
from scipy.stats import norm, multivariate_normal
sys.path.append('./hw1')
import heat_rod as hr

def MetropolisHastingsMCMC(q0, J_func, r_calc, M, rng_seed=None):
    """
    General Metropolis-Hastings MCMC function

    Parameters:
    q0: initial sample
    J_func: proposal distribution function J(q* | q_{k-1})
    r_calc: function to compute acceptance ratio r(q*, q_{k-1})
    M: number of samples to generate
    rng_seed: random seed for reproducibility

    Returns:
    samples: array of MCMC samples
    acceptance_rate: acceptance rate of the chain
    """
    if rng_seed is not None:
        np.random.seed(rng_seed)

    q_current = np.array(q0, dtype=float)
    samples = np.zeros((M, len(q0)))
    samples[0] = q_current

    accepted = 0

    print("Running Metropolis-Hastings MCMC...")
    for i in range(1, M):
        # Step 2.1: Generate proposal from J(q* | q_{k-1})
        q_proposed = J_func(q_current)

        # Step 2.2: Compute the ratio r(q*, q_{k-1})
        r = r_calc(q_proposed, q_current)

        # Step 2.3: Accept with probability min(1, r)
        if np.random.random() < min(1, r):
            q_current = q_proposed
            accepted += 1

        samples[i] = q_current

        if (i + 1) % 200 == 0:
            print(f"    Completed {i + 1}/{M} iterations, acceptance rate: {accepted/(i):.3f}")

    acceptance_rate = accepted / (M - 1)

    return samples, acceptance_rate

def setup_heat_equation_problem(mat_file='HW06_Problem1.mat'):
    """
    Set up the heat equation problem using data from MATLAB file
    """
    data = loadmat(mat_file)

    obs_data = data['observations'].flatten()
    print(f"Shape of observations: {obs_data.shape}")
    T_obs = obs_data

    # Observation locations (as given in the problem)
    x0 = 10 # cm
    dx = 4 # spacing
    x_obs = x0 + np.arange(len(obs_data)) * dx

    # Rod parameters
    a = 0.95 # cm
    b = 0.95 # cm
    k = 2.3 # W/cm/C
    Tamb = 21.29 # C
    L = 70 # cm

    q0 = np.array([-15.0, 0.002]) # [Phi, h]
    phi0, h0 = q0

    rod = hr.SteadyStateRod(a, b, phi0, k, h0, Tamb, L)

    # Known error variance
    sigma2_0 = 4.0 # (2 C)^2

    return rod, x_obs, T_obs, sigma2_0, q0

def create_posterior_ratio_func(rod, x_obs, T_obs, sigma2_0):
    """
    Create function to compute r(q*, q_{k-1}) = \Pi(q*|O_\mu) / \Pi(q_{k-1}|O_\mu)
    For Metropolis algorithm with symmetric proposal, this is the acceptance ratio
    """
    # Prior parameters
    prior_Phi_mean = -15
    prior_Phi_std = 10
    prior_h_mean = 0.001
    prior_h_std = 0.005

    def posterior_ratio(q_star, q_prev):
        """
        Compute r(q*, q_{k-1}) = [\Pi(\Pi_{...}|q*)\Pi\theta_0(q*)] / [\Pi\theta(\Pi_{...}|q_{k-1})\Pi\theta_0(q_{k-1})]
        """
        # Evaluate at q_star
        Phi_star, h_star = q_star
        rod.Phi_val = Phi_star
        rod.h_val = h_star
        T_model_star = rod.Ts(x_obs)

        # Log likelihood for q_star (using log to avoid underflow, then exponentiate)
        n = len(x_obs)
        log_likelihood_star = -0.5 * np.sum((T_obs - T_model_star)**2 / sigma2_0)

        # Log prior for q_star
        log_prior_star = (norm.logpdf(Phi_star, prior_Phi_mean, prior_Phi_std) +
                           norm.logpdf(h_star, prior_h_mean, prior_h_std))

        # Evaluate at q_prev
        Phi_prev, h_prev = q_prev
        rod.Phi_val = Phi_prev
        rod.h_val = h_prev
        T_model_prev = rod.Ts(x_obs)

        # Log likelihood for q_prev
```

```

        log_likelihood_prev = -0.5 * np.sum((T_obs - T_model_prev)**2 / sigma2_0)

# Log prior for q_prev
log_prior_prev = (norm.logpdf(Phi_prev, prior_Phi_mean, prior_Phi_std) +
                  norm.logpdf(h_prev, prior_h_mean, prior_h_std))

# Log ratio
log_ratio = (log_likelihood_star + log_prior_star) - (log_likelihood_prev + log_prior_prev)

# Return actual ratio (exponentiate)
return np.exp(log_ratio)

return posterior_ratio

def run_metropolis_for_heat_equation(D =None, M = None, data_name = None):
    """
    Run Metropolis algorithm for the heat equation problem (Problem 1.1)
    """
    if data_name is None:
        data_name = 'HW06_Problem1.mat'

# Setup problem
rod, x_obs, T_obs, sigma2_0, q0 = setup_heat_equation_problem(data_name)
if D is None:

    D = np.array([[1e-2, 0],
                  [0, 2e-10]])

# Create proposal function  $J(q^* | q_{k-1}) = N(q_{k-1}, D)$ 
def proposal_func(q_current):
    return np.random.multivariate_normal(q_current, D)

# Create function to compute  $r(q^*, q_{k-1})$ 
posterior_ratio_func = create_posterior_ratio_func(rod, x_obs, T_obs, sigma2_0)

# Test at initial point
initial_ratio = posterior_ratio_func(q0, q0) # Should be 1.0
print(f"Initial test -  $r(q_0, q_0) = \{initial\_ratio:.6f\}$ ")

if M is None:
    M = 1000
print(f"\nRunning Metropolis algorithm for {M} iterations...")
print(f"Proposal distribution:  $N(q\_prev, D)$  with  $D = \text{diag}(\{D[0,0]\}, \{D[1,1]\})$ ")

samples, acceptance_rate = MetropolisHastingsMCMC(
    q0,
    proposal_func,
    posterior_ratio_func,
    M,
    rng_seed=42
)

print(f"\nFinal acceptance rate: {acceptance_rate:.3f}")

# Find MAP estimate
# We need to recompute the posterior (likelihood * prior) for each sample
print("\nComputing posterior values for MAP estimate...")

# Prior parameters
prior_Phi_mean = -15
prior_Phi_std = 10
prior_h_mean = 0.001
prior_h_std = 0.005

# Compute unnormalized posterior for each sample
posterior_values = np.zeros(M)
for i, q in enumerate(samples):
    Phi, h = q
    rod.Phi_val = Phi
    rod.h_val = h
    T_model = rod.Ts(x_obs)

# Likelihood
n = len(x_obs)
likelihood = np.exp(-0.5 * np.sum((T_obs - T_model)**2 / sigma2_0))

# Prior
prior = (norm.pdf(Phi, prior_Phi_mean, prior_Phi_std) *
         norm.pdf(h, prior_h_mean, prior_h_std))

# Unnormalized posterior ( $\pi(\text{data}|q)\pi_0(q)$ )
posterior_values[i] = likelihood * prior

# Find MAP estimate (maximum unnormalized posterior)
map_idx = np.argmax(posterior_values)
q_map = samples[map_idx]
post_map = posterior_values[map_idx]

print(f"\n{'='*60}")
print("PROBLEM 1.1 RESULTS")
print(f"{'='*60}")
print(f"\nMAP Estimate ( $q_{MAP}$ ):")
print(f"  O| = {q_map[0]:.6f} W/cmBI")
print(f"  h = {q_map[1]:.6f} W/cmBI/B°C")
print(f"   $\Pi(\Pi_{...}|q_{MAP})\Pi_0(q_{MAP}) = \{post\_map:.2e\}$ ")

print(f"\nPosterior Distribution  $\Pi(q|\Pi_{...})$  Summary:")
print(f"{'='*60}")
print(f"O|:")
print(f"  Mean = {np.mean(samples[:, 0]):.6f} W/cmBI")
print(f"  Std = {np.std(samples[:, 0]):.6f} W/cmBI")
print(f"  95% CI = [{np.percentile(samples[:, 0], 2.5):.6f}, {np.percentile(samples[:, 0], 97.5):.6f}])")

print(f"\nh:")
print(f"  Mean = {np.mean(samples[:, 1]):.6e} W/cmBI/B°C")
print(f"  Std = {np.std(samples[:, 1]):.6e} W/cmBI/B°C")
print(f"  95% CI = [{np.percentile(samples[:, 1], 2.5):.6e}, {np.percentile(samples[:, 1], 97.5):.6e}])

fig, axes = plt.subplots(2, 2, figsize=(14, 12))

# Plot 1: Phi histogram (top left)
axes[0, 0].hist(samples[:, 0], bins=30, density=True, alpha=0.7, color='skyblue', edgecolor='black')
axes[0, 0].axvline(q_map[0], color='red', linestyle='--', linewidth=2, label=f'MAP: {q_map[0]:.4f}')
axes[0, 0].set_xlabel('O| (W/cmBI)')
axes[0, 0].set_ylabel('Density')
axes[0, 0].set_title('Marginal Posterior  $\Pi(O|\Pi_{...})$ ')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# Plot 2: h histogram (top right)
axes[0, 1].hist(samples[:, 1], bins=30, density=True, alpha=0.7, color='lightcoral', edgecolor='black')
axes[0, 1].axvline(q_map[1], color='red', linestyle='--', linewidth=2, label=f'MAP: {q_map[1]:.6f}')

```

```
axes[0, 1].set_xlabel('h (W/cmBI/B°C)')
axes[0, 1].set_ylabel('Density')
axes[0, 1].set_title('Marginal Posterior  $\Pi(h|\Pi_{\dots})$ ')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# Plot 3: Joint posterior scatter (bottom left)
axes[1, 0].scatter(samples[:, 0], samples[:, 1], alpha=0.5, c='purple', s=10)
axes[1, 0].plot(samples[:, 0], samples[:, 1], alpha=0.5, c='purple')
axes[1, 0].scatter(q_map[0], q_map[1], color='Magenta', s=200, marker='*', label='MAP')
axes[1, 0].set_xlabel('Oi (W/cmBI)')
axes[1, 0].set_ylabel('h (W/cmBI/B°C)')
axes[1, 0].set_title('Joint Posterior Samples  $\Pi(O_i, h|\Pi_{\dots})$ ')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# Plot 4: Model fit with MAP parameters (bottom right)
rod.Phi_val = q_map[0]
rod.h_val = q_map[1]
T_map = rod.Ts(x_obs)

axes[1, 1].plot(x_obs, T_obs, 'ko', markersize=8, label='Observations')
axes[1, 1].plot(x_obs, T_map, 'r-', linewidth=2, label='MAP prediction')
axes[1, 1].set_xlabel('Position x (cm)')
axes[1, 1].set_ylabel('Temperature (B°C)')
axes[1, 1].set_title('Model Fit with MAP Parameters')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('problem1.1_results.png', dpi=150)
plt.show()
return samples, q_map, posterior_values, acceptance_rate
```